

## **High Level Description**

Our synthesizer was implemented using two oscillators that produced square and saw waves. Our polyphonic synthesizer supports up to 5 voice outputs, meaning it can play a combination of 5 notes simultaneously. The synthesizer comes equipped with both a high and low pass filter, though the high pass has a major bug where it acts more as a distorter than a high pass filter, due to its conflicting position with the low pass filter. Users can switch between the square and saw wave or use both. We intended to have a full range of C to B, but currently only C - A works. This is likely due to a breadboard issue, as moving it around results in some output. The synth also comes with an octave knob that allows the user to switch between different octaves for their keyboard from C2 to C6. Our synth comes equipped with a screen displaying all possible adjustments that could be made. We intended for the screen to actively output information parameters; however, we were unable to do so without causing conflicts with the audio output.

## **Elements of Complexity**

### **FPGA Based Audio Synthesis**

Our project required most of the work to be done in Verilog, with our FPGA handling the digital signal processing and synthesis. The FPGA handled oscillator generation, including square and saw waves, pitch control, filters, ADSR envelope behavior, and note selection. We were able to create a modular synthesizer using Verilog without the help of the Pico or Pico libraries. Many guides often used the Pico as the audio processor and synthesizer itself, with the FPGA just being used for audio output. In our case, the FPGA was the synthesizer itself, streaming 8-bit information to the Pico for output.

### **User input handling on the FPGA**

Having to handle many moving parts with an FPGA proved to be difficult. Our initial intention was to send signals to the FPGA via the Pico. But issues regarding MISO and MOSI required us to have the user input be handled directly on the FPGA as well. Our project uses rotary encoders and many buttons. These inputs needed debouncing, direction detection, mode switching, octave shifting, and parameter control. There was also a learning curve regarding having to learn how rotary encoders are handled in Verilog and on the FPGA.

### **Real time audio streaming and SPI communication**

Our initial intention was to simply save audio to a folder with no real time audio being output. Having us play in an allocated amount of time, recording that play, and saving it to an output folder. We decided to instead output our audio from our computer speakers, which meant having to stream the 8-bit audio directly to the Pico and then to the computer. The FPGA generates audio continuously, the Pico receives it, and the computer plays it live. This proved to be difficult because dropped or delayed samples can cause glitches, silence, or distortion, which

in our final version sometimes still occurs, especially under rapid changes to the envelope or filters.

Furthermore, this came under resource management as well, having to balance mixing and volume so that the signal being streamed to the pico wouldn't be clipped and distorted, as well as managing the memory constraints for the pico, especially when implementing polyphonic play. As our pico had to communicate with a second FPGA, it meant having to balance our project structure.

### **Pico to computer audio output**

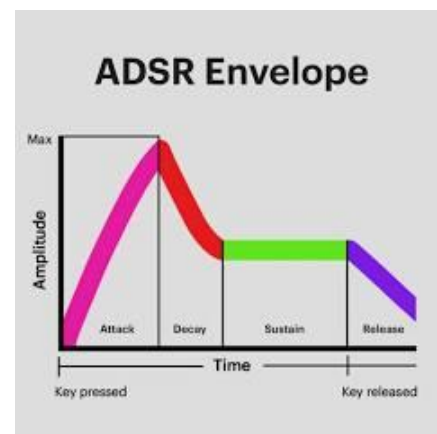
Though not as complex as the other components, the difficulty existed in the learning curve of figuring out how USB transmission occurs with the Pico. This meant having to write a personal Python script made to handle speaker output of data information.

## **User Guide**

The left-most knob is used for octave control. Turning left goes down an octave, and turning right goes up an octave. The second knob is our low pass filter, turning right begins to cut off the signal being sent to the pico, and turning right gradually returns the frequencies back to normal. The same occurs for the third knob, which is our high pass filter. Turning right cuts high frequencies, and turning left returns those frequencies to normal. The fourth knob can only be used during ADSR mode. The button to the left of the breadboard turns on ADSR mode. To which all knobs are then used to configure the Attack, Decay, Sustain and Release of our synthesizer. The pitch knob now controls the Attack by turning right, we increase the time it takes for our oscillator to reach maximum amplitude. Turning right decreases the time necessary. Our low pass filter now handles the decay on our synth, allowing us to create a change in amplitude after the initial attack; the orientation of which we turn is the same as our attack. Our high pass filter becomes the sustain, as in how long we can hold onto that note while pressing it. Our fourth knob is now used to control the release or how long it takes for our synth to reach zero amplitude. We have two button switches to the right. The left button turns the Square wave on, the right button turns the Saw wave on, and both waves can be on at the same time. These modes can be used to affect one another and can result in complex sound design, from actual synthesizers to even drum sound synthesis.

## **List of Hardware**

- 4 x Rotary Encoders
- 15 x Button Switches
- 2 x Icesugar Pro FPGA's
- 1 x Raspberry Pi Pico 2
- 1 x LCD Display
- 1 x MacBook Air



## List of Libraries

- LVGL
  - Serves to streamline the UI design of our LCD Display through SPI from the Pico
- Pico SDK libs, especially Pico USB/stdio libraries
  - Used so the Pico could send audio data to the computer over USB serial and for SPI communication to the second FPGA
- Pyserial
  - Used in the computer side live audio script to read the audio stream coming from the Pico
- Sounddevice
  - Used to output the streamed audio live through the computer speakers
- OSS CAD Suite tools
  - Used to synthesize and program the FPGA design

## Protocols

- Rotary encoder quadrature signaling
  - Used by the knobs to detect clockwise/counterclockwise movement
- PCM audio format
  - Used for representing raw audio samples and digital information
- SPI
  - Main communication between the FPGA and Pico for audio samples. Also used between Pico and the LCD FPGA.
- USB Serial / CDC
  - Pico sends the audio stream to the computer over USB

## How Requirements Were Met

We are using two Icesugar Pros for this project. The main FPGA that meets the first requirement is used to process and synthesize digital audio samples in accordance with event driven parameters made by the user, which satisfies the event driven execution requirement. The project uses a mixture of real time execution and event driven behavior. The FPGA continuously generates audio samples, while user inputs such as buttons, rotary encoders, SPI byte arrivals, and timed LCD refreshes act as events that change the synth parameters or output state. For example, when the user presses/releases a key, an event occurs that moves the envelope into attack, decay, sustain, or release stages. The Pico requirement is handled by having the Pico be in charge of USB communication and unloading of the audio sample so that they can output to the computer speakers. The Pico also handles SPI communication to the second FPGA for the LCD Display. LCD display requirement is met by having the LCD display the visual representation of our synthesizer, showcasing all possible features contained in it. However, it does actively

change in real time according to the user parameters; this was a feature of our project that we failed to meet, so the LCD only shows a static UI.

Wiring diagram

